Prueba técnica (48 -72 horas) — Solutions Engineer (AI Agents)

Contexto

En este rol vas a ayudar a clientes a **diseñar, configurar y desplegar agentes de IA** orientados a comunicación/operaciones. Buscamos ver tu capacidad para:

- Entender un caso de uso y transformarlo en una solución funcional.
- Aplicar **prompt engineering** (estructura, reglas, tono, guardrails, fallback).
- Iterar y mejorar la solución con base en pruebas.
- Documentar y dejar "handoff" listo (como si pasara a Customer Success).
- Construir algo reutilizable (templates/configuración por cliente).
- (Opcional) Integrar o simular integraciones (API/webhook) de forma clara.

Objetivo

Construir un **mini "kit de despliegue"** de un agente conversacional (MVP) que resuelva un caso operativo de logística y que pueda:

1. Atender solicitudes del usuario por chat
2. Hacer preguntas de aclaración cuando falten datos
3. Consultar/actualizar información vía un **Mock API** (que tú mismo construyes)
4. Mantener un tono/estilo consistente configurado por "cliente"
5. Evitar alucinaciones (si no sabe, lo dice)
6. Incluir fallback/escalamiento (cuando el caso no aplica o falta información)

Caso de uso

El agente ayudará a un usuario a gestionar envíos. Debe soportar **mínimo 3 intenciones**:

1. **Consultar estado de envío**
2. **Reprogramar entrega**

3. **Crear un ticket/incidencia** (daño, retraso, pérdida, etc.)

Ejemplos de mensajes

- "¿Cuál es el estado del envío 12345?"
- "Necesito mover la entrega del envío 12345 para mañana en la tarde."
- "Mi paquete llegó abierto, quiero reportarlo."

Requisitos funcionales (obligatorios)

A. Conversación y comportamiento

- El agente debe:
    - Identificar la intención del usuario.
    - Solicitar información faltante (ej. shipment_id, nueva fecha, motivo).
    - Responder con claridad, tono consistente y formato legible.
    - Tener reglas explícitas de no inventar datos.

B. Mock API (obligatorio)

Debes crear un **Mock API local** (puede ser FastAPI/Flask/Express) con al menos estos endpoints:

1. GET /shipments/{id}
   Retorna información del envío (estado, ETA, dirección, historial, etc.)
2. POST /shipments/{id}/reschedule
   Recibe nueva fecha/hora/ventana y devuelve confirmación (o error si no se puede)
3. POST /tickets
   Crea un ticket con: shipment_id, tipo, descripción, severidad, contacto

   Puedes persistir en memoria o en un archivo JSON. No hace falta base de datos.

C. Orquestación "tool use"

El agente debe "usar herramientas" de forma visible en el flujo:

- Si el usuario pregunta estado: llama al GET /shipments/{id}.
- Si reprograma: valida datos → llama a POST /shipments/{id}/reschedule.
- Si crea incidencia: valida datos → llama a POST /tickets.

(No necesitas un framework complejo de tool-calling; puede ser una lógica tuya que detecte cuándo llamar a cada endpoint.)

D. Templates / configuración por cliente (obligatorio)

Debes permitir configurar el agente con un archivo por cliente (YAML/JSON), por ejemplo:

- tone: formal / cercano
- language: es
- policies: no inventar, pedir confirmación, escalar cuando aplique
- required_fields: por intención
- message_formats: plantillas para respuesta (status_update, delay_notice, etc.)

- Incluye al menos **1 template** y ojalá 2 (Cliente A / Cliente B) para demostrar variación de tono/reglas.

savant®

Requisitos no funcionales (obligatorios)

- Todo debe correr con **herramientas gratuitas**.
- El evaluador debe poder ejecutar tu solución en **≤ 10 minutos** siguiendo tu README.
- Debes usar el json compartido que contiene informacion de ordenes de entrega.
- El proyecto debe tener estructura limpia, instrucciones claras y ejemplos.

Herramientas permitidas (gratuitas)

Elige lo que prefieras, pero debe ser gratis.

Recomendadas (100% gratis, local):

- LLM local: **Ollama** (Llama/Mistral)
- Backend: **Python (FastAPI/Flask)** o **Node (Express)**
- UI: **Streamlit** o una web simple

Importante:

- Si usas un LLM en la nube, debe ser **free tier real** (sin necesidad de tarjeta/pago) y tu proyecto debe funcionar sin costos para evaluarlo.

Entregables (lo que debes entregar)

1) Repositorio (GitHub) o entrega fisica de archivos

Con al menos:

- Código del agente
- Código del Mock API
- Templates (YAML/JSON)
- Documentación

2) README.md (obligatorio)

Debe incluir:

- Requisitos (versiones)
- Instalación
- Cómo correr Mock API
- Cómo correr el agente/UI
- Cómo probar (ejemplos de prompts y resultados esperados)
- Troubleshooting rápido

3) Documentos (obligatorio)

En carpeta `/docs` o raíz:

- `SOW.md` (Statement of Work corto: alcance, supuestos, fuera de alcance)
- `RUNBOOK.md` (operación: cómo reiniciar, dónde ver logs, errores comunes)
- `PROMPT_NOTES.md` (iteración y pruebas)

4) Evidencias (obligatorio)

- sample_conversations.md con al menos 3 conversaciones:

    1. flujo “feliz”
    2. faltan datos (agente pregunta)
    3. caso límite (error API / envío no existe / reprogramación inválida)

5) Video corto (recomendado, máximo 3 minutos)

Grabación de pantalla mostrando:

- 1 conversación exitosa con consulta de estado
- 1 conversación con reprogramación
- 1 fallback (no tiene datos / escalamiento / error)

savant®